

Broken Access Control (BAC) di CTF Jeopardy

Broken Access Control adalah celah keamanan ketika aplikasi gagal membatasi akses user terhadap resource tertentu.

Akibatnya, attacker bisa:

- Mengakses data user lain
- Menjadi admin
- Membuka file rahasia
- Mengubah data yang bukan miliknya
- Mengakses endpoint tersembunyi

Di dunia nyata, BAC termasuk vulnerability paling berbahaya dalam OWASP Top 10.

Konsep Dasar Access Control

Normalnya aplikasi harus memeriksa:

Role Hak Akses

Guest hanya melihat public page

User data milik sendiri

Admin seluruh data & fitur

Jika validasi gagal → terjadi Broken Access Control.

Jenis-Jenis Broken Access Control

1. Vertical Privilege Escalation

User biasa bisa menjadi admin.

Contoh:

/admin

Harusnya hanya admin.

Tetapi server tidak mengecek role.

2. Horizontal Privilege Escalation

User bisa mengakses data user lain.

Contoh:

`/profile?id=1`

Diubah menjadi:

`/profile?id=2`

Lalu data user lain muncul.

3. Forced Browsing

Mengakses endpoint tersembunyi langsung.

Contoh:

`/admin`

`/debug`

`/backup.zip`

4. Insecure Direct Object Reference (IDOR)

Object internal dapat diakses langsung tanpa validasi.

Contoh:

`/download?file=flag.txt`

Contoh Broken Access Control di CTF Jeopardy

Contoh 1 — IDOR

Soal

Website memiliki fitur profile.

URL:

`/profile?id=1`

Response:

Welcome Asep

Hint challenge:

“Can you see another user?”

Langkah Exploit

Ubah parameter:

/profile?id=2

Response:

Welcome admin

FLAG{IDOR_SUCCESS}

Kenapa Bisa Terjadi?

Server hanya mengambil data berdasarkan ID:

```
SELECT * FROM users WHERE id=?
```

Tetapi tidak memeriksa:

```
if session_user.id != requested_id:
```

```
    deny()
```

Simulasi Source Code Vulnerable

```
@app.route('/profile')
```

```
def profile():
```

```
    user_id = request.args.get("id")
```

```
    conn = sqlite3.connect("database.db")
```

```
    cur = conn.cursor()
```

```
    cur.execute(
```

```
        "SELECT username FROM users WHERE id=?",
```

```
(user_id,)  
)
```

```
user = cur.fetchone()
```

```
return f"Welcome {user[0]}"
```

Masalah:

- Tidak ada validasi ownership
- Semua ID bisa diakses

Fix Aman

```
@app.route('/profile')
```

```
def profile():
```

```
    requested_id = request.args.get("id")
```

```
    if str(session["user_id"]) != requested_id:
```

```
        return "Forbidden", 403
```

Contoh 2 — Admin Panel Bypass

Soal

Login sebagai user biasa:

username: guest

password: guest

Setelah login hanya ada:

/home

Tetapi hint:

“Admins hide secrets well.”

Enumerasi

Coba akses manual:

/admin

Ternyata terbuka:

FLAG{ADMIN_PANEL_EXPOSED}

Penyebab

Developer hanya menyembunyikan tombol admin di frontend:

```
<!-- tombol admin dihapus -->
```

Tetapi backend tetap bisa diakses.

Contoh Vulnerable Backend

```
@app.route('/admin')
```

```
def admin():
```

```
    return open("flag.txt").read()
```

Tidak ada pengecekan role.

Fix Aman

```
@app.route('/admin')
```

```
def admin():
```

```
    if session.get("role") != "admin":
```

```
        return "Forbidden", 403
```

```
    return open("flag.txt").read()
```

Contoh 3 — Horizontal Privilege Escalation

Soal

User dapat melihat invoice:

/invoice/1001

Hint:

"Invoices are incremental."

Exploit

Coba:

/invoice/1002

/invoice/1003

Salah satu response:

FLAG{INVOICE_LEAK}

Contoh 4 — Cookie Role Manipulation

Soal

Cookie:

role=user

Hint:

"Cookies can lie."

Exploit

Ubah cookie menjadi:

role=admin

Server:

```
if request.cookies.get("role") == "admin":
```

```
    return flag
```

Response:

FLAG{COOKIE_ADMIN}

Tools yang Sering Dipakai di CTF

Burp Suite

Untuk:

- Intercept request
 - Modify parameter
 - Ubah cookie
 - Repeater
 - Intruder
-

Browser DevTools

Untuk:

- Edit cookie
 - LocalStorage
 - SessionStorage
-

ffuf

Directory fuzzing:

```
ffuf -u http://target/FUZZ -w wordlist.txt
```

Mencari:

- /admin
 - /backup
 - /dashboard
-

Cara Berpikir Saat Menemukan BAC di CTF

Selalu tanyakan:

“Apa yang terjadi jika saya mengganti...”

- ID
- Username

- Cookie
 - Role
 - Endpoint
 - HTTP Method
-

“Apakah backend benar-benar memvalidasi?”

Karena:

- Hidden button ≠ secure
 - Disabled form ≠ secure
 - Frontend check ≠ secure
-

Checklist Exploit BAC di CTF

Coba akses:

/admin

/dashboard

/api/admin

/hidden

/debug

Coba ubah:

?id=1 → ?id=2

/user/1 → /user/2

Coba edit cookie:

role=user → admin

isAdmin=false → true

Coba HTTP Method:

GET /admin

POST /admin

PUT /admin

Kadang hanya satu method yang diproteksi.

Kombinasi BAC dengan Vulnerability Lain

BAC sering digabung dengan:

Vulnerability	Kombinasi
----------------------	------------------

SQL Injection	bypass auth lalu akses admin
---------------	------------------------------

XSS	mencuri session admin
-----	-----------------------

SSRF	akses internal admin panel
------	----------------------------

File Upload	upload shell ke area admin
-------------	----------------------------

JWT Misconfig	ubah role jadi admin
---------------	----------------------

Contoh Challenge Mini

Scenario

Login:

guest:guest

URL:

/account?user=guest

Response:

No flag here

Analisis

Kemungkinan:

- parameter user dapat diganti
-

Exploit

/account?user=admin

Response:

FLAG{BROKEN_ACCESS_CONTROL}

Tips Penting Saat CTF

BAC biasanya:

- tidak terlalu teknis
- lebih ke logic flaw
- membutuhkan observasi
- sering tersembunyi di parameter kecil

Karena itu:

- baca semua request
 - perhatikan response berbeda
 - cek cookie/session
 - brute force endpoint seperlunya
-

Ringkasan

Jenis	Contoh
IDOR	ubah ID user
Admin Bypass	akses /admin langsung
Horizontal Escalation	akses data user lain
Cookie Manipulation	ubah role
Forced Browsing	buka endpoint tersembunyi

Alur Umum Exploit BAC di CTF

Login/Register



Intercept Request



Ubah Parameter/Role/ID



Bypass Authorization



Dapat Flag